THEMAJORD'HOME

Guia técnico
para o lançamento

Site, painel de administração, reservas (Beds24 + Stripe),
Google Ads e publicação na web — o que está feito,
o que está preparado e o que falta fazer.

Destinatário: engenheiro responsável pela publicação e integrações
Data: 2 de novembro de 2026
Repositório: github.com/Jesunilson-Mateus/Project-Themjordhomewebsite (branch main)
Documentos relacionados no repositório: docs/INTEGRACAO-BEDS24-STRIPE.md · README.md

Conteúdo

1. Resumo e estado atual
2. O código e a estrutura do projeto
3. Painel de administração
4. Reservas: Beds24 + Stripe
5. Publicar, atualizar e cópias de segurança
6. SEO técnico (antes dos anúncios)
7. Google Ads e medição
8. Pendentes e pontos de atenção
9. Checklist de lançamento

Como ler este documento. As secções 1–3 explicam o que existe. A secção 4 é o trabalho de ligação às contas Beds24 e Stripe. As secções 5–7 cobrem a publicação, o SEO e o Google Ads. A secção 8 lista o que **ainda não está resolvido** — inclui pontos legais que devem ser tratados antes de aceitar pagamentos.

1. Resumo e estado atual

O site da TheMajord'Home é um site de alojamento local no Porto (60 apartamentos) com páginas de serviços para proprietários (gestão, concierge, limpeza, manutenção). É um site estático em HTML/CSS/JS, com um painel de administração em PHP para a equipa editar conteúdos, e uma API de reservas em PHP preparada para o Beds24 e o Stripe, atualmente **desligada** (modo de demonstração).

Componente	Estado	Notas
Site público (PT / EN / FR)	**Pronto**	14 páginas, tema claro/escuro, adaptado a telemóvel (verificado de 320 a 1280 px).
Pesquisa na coleção	**Pronto**	Nome, zona e nº de hóspedes (1–10), na página inicial e em apartments.html.
Painel de administração	**Pronto**	Apartamentos (ativo/inativo), fotos, comodidades, galerias dos serviços, avaliações, utilizadores, modo manutenção. Precisa de PHP.
Páginas 404 e manutenção	**Pronto**	404.html e manutencao.html (PT/EN/FR), ligadas pelo .htaccess da raiz (Apache). Testadas com Apache.
Dados (data/*.json)	**Pronto**	Fonte de verdade. O painel e scripts/sync-data.js regeneram os ficheiros .js.
Reservas — Beds24	**Preparado, desligado**	API em api/. Falta: credenciais, mapa de quartos, confirmar 4 respostas da API.
Pagamentos — Stripe	**Preparado, desligado**	Checkout + webhook prontos e testados com serviço simulado. Falta: chaves.
Emails de reserva	**Básico**	Usa mail() do PHP. Trocar por SMTP/serviço transacional.
Google Ads / Analytics	**Por fazer**	O aviso de cookies já expõe o consentimento (window.TMHConsent).
SEO técnico	**Por fazer**	Sem meta description, favicon, robots.txt, sitemap.
Textos legais	**Por refazer**	Aviso legal é um modelo francês. Faltam privacidade, cookies, termos de reserva.
Alojamento e domínio	**Por decidir**	Tem de ser alojamento com PHP 8.1+. Não serve Netlify.

Alojamento: o site precisa de PHP (painel, API de reservas, página 404 e modo manutenção). Alojamentos só de ficheiros estáticos (p. ex. Netlify, GitHub Pages) não servem.

2. O código e a estrutura do projeto

Onde está

- Repositório: `github.com/Jesunilson-Mateus/Project-Themjordhomewebsite` — branch `main` (público).
- A cópia de trabalho é a do repositório. Existe uma cópia antiga em Downloads\majordhome-project (setembro) que **não deve ser usada**, exceto para recuperar o ficheiro admin/LEIA-ME.txt (ver secção 3).
- Não há passo de compilação (build): os ficheiros são publicados tal como estão.

Pastas principais

Pasta / ficheiro	Conteúdo
`*.html`	Páginas: index, apartments, property (modelo de cada apartamento), search-results, proprietarios, servico-*, reserva, legal-notice; review.php (avaliações).
`css/style.css`	Todo o estilo do site (variáveis de tema no :root).
`js/`	main.js (página inicial, coleção, pesquisa), property.js, booking-*.js (reservas), i18n.js (traduções), cookie-consent.js, map.js.
`data/`	Fonte de verdade: properties.json, translations.json, amenity-*.json, nearby-labels.json, service-galleries.json, beds24-rooms.json, reviews.json. Protegida por .htaccess.
`img/`	Fotos de cada apartamento (img/<slug>/) e dos serviços (img/servicos/).
`admin/`	Painel de administração (PHP).
`api/`	API de reservas (PHP): Beds24 + Stripe.
`scripts/sync-data.js`	Regenera os .js e as galerias dos serviços a partir dos JSON.
`docs/`	Este guia e o detalhe técnico da integração Beds24/Stripe.

Regra dos dados

Os ficheiros em `data/` são a fonte de verdade. O site lê cópias embutidas em `js/properties-data.js`, `js/i18n.js` e nas galerias das páginas `servico-*.html`. O painel atualiza tudo sozinho; quem editar os JSON à mão tem de correr:

```
node scripts/sync-data.js          # atualiza os ficheiros gerados
node scripts/sync-data.js --check  # só verifica (útil em CI)
```

O script recusa-se a correr se a sincronização fosse apagar entradas que só existem nos .js.

Correr localmente

```
php -S 127.0.0.1:8080 -t .      # na raiz do projeto; abrir http://127.0.0.1:8080
```

Abrir os ficheiros com duplo clique também mostra o site, mas o painel e a API só funcionam com PHP.

3. Painel de administração

Acessível em `/admin/`. Permite à equipa, sem mexer em código:

- editar cada apartamento: nome, capacidade, descrições (PT/EN/FR), comodidades, regras da casa;
- gerir fotos (enviar, ordenar, escolher a capa, apagar) — redimensionadas para 1800 px;
- editar as galerias "Em imagens" das 4 páginas de serviços (separador Serviços);
- aprovar avaliações de hóspedes e gerar links de convite (review.php);
- ativar / desativar apartamentos: um inativo desaparece do site e não pode ser reservado (dados guardados);
- ligar / desligar o **modo manutenção** (separador Manutenção): os visitantes passam a ver manutencao.html;
- gerir utilizadores do painel.

Credenciais

> O ficheiro `admin/LEIA-ME.txt` tem as instruções e a password da primeira conta. Está fora do git (correto) e **só existe na cópia antiga em Downloads**. Deve ser entregue ao engenheiro por um canal seguro (não por email simples). O ficheiro `admin/users.json` é criado no servidor; a password inicial deve ser trocada no primeiro acesso.

Requisitos no servidor

- PHP 8.1+ com as extensões **gd** (fotos), **curl** (API), **mbstring**.
- O PHP tem de poder escrever em: `data/`, `data/backups/`, `data/reservations/`, `img/`, `admin/users.json`, `js/properties-data.js`, `js/i18n.js`, `js/reviews-data.js` e nas 4 páginas `servico-*.html`.
- Cópias de segurança automáticas antes de cada gravação: todos os data/*.json e as contas do painel, em `data/backups/AAAA-MM-DD_HHMMSS/` (as últimas 60) — ver secção 5.
- Proteções existentes: `admin/.htaccess` (users.json e .txt/.md), `data/.htaccess` (pasta inteira), `api/.htaccess` (config e bibliotecas). Em **nginx** estas regras têm de ser recriadas.
- Recomendado: só por HTTPS; considerar restringir /admin/ por IP ou com autenticação extra.

4. Reservas: Beds24 + Stripe

A integração está construída e testada contra serviços simulados, mas desligada. O detalhe completo (contrato da API, códigos de erro, exemplos JSON) está em `docs/INTEGRACAO-BEDS24-STRIPE.md`.

Como funciona uma reserva

1. O calendário pede as noites ocupadas ao servidor, que as lê do Beds24.
2. O resumo mostra o preço calculado pelo Beds24 — **sempre no servidor**, nunca vindo do browser.
3. Ao confirmar, o servidor cria a reserva no Beds24 em estado request (bloqueia as datas) e abre uma página de pagamento Stripe Checkout (válida 30 min).
4. O Stripe chama o webhook; a reserva passa a confirmed no Beds24 e são enviados emails à equipa e ao hóspede.
5. Se o cliente não pagar a tempo, o webhook de expiração cancela a reserva e liberta as datas.
6. O cliente volta para `reserva.html`, que mostra o estado (pendente / confirmada / cancelada).

Ficheiros

Ficheiro	Função
`js/booking-config.js`	Interruptor do site: mode 'mock' (atual) ou 'live'.
`js/booking-api.js`	Única camada do browser que fala com a API.

Ficheiro	Função
`api/config.example.php`	Modelo → copiar para api/config.php no servidor (fora do git).
`api/beds24.php`	Cliente Beds24 v2: autenticação pronta; 4 pontos marcados VERIFICAR.
`api/stripe.php`	Checkout Session e verificação da assinatura do webhook (pronto).
`api/*.php`	Endpoints: availability, search, quote, reservations, stripe-webhook.
`data/beds24-rooms.json`	Mapa apartamento → roomId do Beds24 (60 entradas por preencher).

Passos — Beds24

1. Beds24 → Settings → Apps & Integrations → API: criar um invite code (leitura de inventário e propriedades; leitura/escrita de reservas). Expira em 24 h.
2. Trocar o código por um refresh token (uma vez) em `/authentication/setup`; guardar em `beds24.refresh_token`. Os tokens de 24 h são gerados e guardados em cache automaticamente. O refresh token expira após 30 dias sem uso.
3. Preencher `data/beds24-rooms.json` com o roomId de cada um dos 60 apartamentos.
4. Confirmar no Swagger (beds24.com/api/v2), com a conta real, o formato das respostas de availability, offers e POST /bookings, e o campo de notas internas usado ao confirmar.
5. Recomendado: registar o pagamento do Stripe na reserva (invoiceItems) para o saldo aparecer pago.

Passos — Stripe

1. Chave secreta (`sk_test_…` para testes, depois `sk_live_…`) em `stripe.secret_key`.
2. Webhook no painel do Stripe: `https://<domínio>/api/stripe-webhook.php` com os eventos `checkout.session.completed` e `checkout.session.expired`; copiar o segredo whsec_… para `stripe.webhook_secret`.
3. `charge_percent`: 100 cobra tudo; 30 cobra um sinal de 30%. `enabled: false` desliga o pagamento online (as reservas ficam como pedido e a equipa confirma por email).
4. Testar com chaves de teste e o cartão 4242 4242 4242 4242 antes de passar às chaves live.

Ligar

1. `api/config.php`: mode = 'live' e site_url correto.
2. `js/booking-config.js`: mode: 'live'.
3. Atualizar o texto `property.finePrint` em js/i18n.js (ainda diz que preços em tempo real "chegam em breve").

Decidir antes de produção

- **Pesquisa:** api/search.php faz 1 pedido ao Beds24 por apartamento (60 por pesquisa); o Beds24 limita a ~100 créditos por 5 min por conta → agrupar pedidos ou usar cache (~10 min).
- **Emails:** trocar mail() por SMTP ou serviço transacional; configurar SPF, DKIM e DMARC no domínio.
- **Anti-spam** no POST de reservas (cada pedido cria uma reserva em espera no Beds24).
- **Reembolsos e cancelamentos:** não há reembolsos automáticos; definir política (ver secção 8).
- **Alternativa mais simples:** usar o motor de reservas do próprio Beds24, que tem integração nativa com o Stripe — perde-se o assistente do site, mas não é preciso a API.

5. Publicar, atualizar e cópias de segurança

Alojamento

- Alojamento com **PHP 8.1+** (gd, curl, mbstring) — p. ex. cPanel/LAMP, como o do site atual themajordhome.com.
- HTTPS obrigatório (Let's Encrypt ou equivalente), com redirecionamento http → https e escolha www / sem www.
- Apache: os .htaccess já existentes funcionam. nginx: recriar as regras de bloqueio (data/, admin/users.json, api/lib|beds24|stripe|config.php, ficheiros .txt/.md do admin) e as do .htaccess da raiz (abaixo).
- O site tem de ficar na **raiz do domínio** (404.html e manutencao.html usam <base href="/">).

O .htaccess da raiz

- `ErrorDocument 404 /404.html`: página para endereços inexistentes (com X-Robots-Tag noindex).
- **Modo manutenção:** se existir `data/maintenance.flag` (criado pelo painel), todo o site público responde com `manutencao.html` e o código 503 + Retry-After. Continuam acessíveis: /admin/, `api/stripe-webhook.php`, css/, js/, img/. Quem liga a manutenção recebe o cookie `tmh_preview=1` e continua a ver o site (conveniência, não é proteção).
- Bloqueia `.git/`, outros ficheiros escondidos (exceto .well-known) e `_arquivo/` — importante se o site for publicado com git clone.
- Requer mod_rewrite e mod_headers. Em nginx: error_page 404/503, um if (-f …/data/maintenance.flag) com as mesmas exceções, e location ~ /\. { return 404; }.

Passos

1. Apontar o domínio (DNS) para o alojamento e ativar HTTPS.
2. Publicar o conteúdo do repositório (p. ex. `git clone` ou cópia por SFTP). **Não publicar** `.claude/`, `_arquivo/` (não estão no git) nem o ficheiro solto ADMIN_PANEL_PREVIEW.html.
3. Criar no servidor (fora do git): `api/config.php` e a conta do painel (admin/LEIA-ME.txt).
4. Dar ao PHP permissão de escrita nas pastas e ficheiros listados na secção 3.
5. Desligar a apresentação de erros do PHP em produção (display_errors = Off) e manter os logs.
6. Testar: páginas, idiomas, painel (/admin/), pesquisa, e — já em modo live com chaves de teste — uma reserva completa até ao webhook.
7. Ligar as chaves live do Stripe e mudar os dois interruptores para 'live'.

Atualizar o site

Depois de o site estar no ar, os dados verdadeiros são os do servidor — a equipa edita-os no painel. Numa atualização, estes ficheiros **não podem ser substituídos** pelas versões do repositório: `data/`, `img/`, `admin/users.json`, `api/config.php`, `js/properties-data.js`, `js/i18n.js`, `js/reviews-data.js` e `servico-*.html`. Se for preciso mexer num deles (p. ex. textos da interface em js/i18n.js), trazer primeiro a versão do servidor para o repositório.

1. No painel, **ligar o modo manutenção**.
2. **Fazer uma cópia** do servidor: `data/` (inclui data/backups/ e data/reservations/), `img/`, `admin/users.json`, `api/config.php`.
3. Publicar os ficheiros novos do código, sem substituir os da lista acima.
4. Testar páginas, línguas, painel, pesquisa e, com as reservas ligadas, uma reserva de teste.
5. No painel, **desligar o modo manutenção**.

Onde estão as cópias de segurança

O quê	Onde e o que contém	Como recuperar
Cópias automáticas do painel	No servidor, `data/backups/AAAA-MM-DD_HHMMSS/`: uma pasta por gravação no painel, as **últimas 60**. Todos os data/*.json (apartamentos, traduções, comodidades, galerias, avaliações, mapa Beds24) e users.json, como estavam **antes** da gravação.	Copiar os ficheiros para data/ (users.json para admin/) e correr node scripts/sync-data.js para regenerar os .js e as galerias (sem Node no servidor: numa cópia local, e enviar os ficheiros gerados).
Código	GitHub — histórico do git.	git log; git checkout <commit> -- <ficheiro>.
Fotos	img/ no servidor. **Sem cópia automática.** Apagar uma foto no painel só a retira do site; o ficheiro fica em img/.	Voltar a escolhê-la no painel; se o ficheiro se perdeu, cópia manual ou do alojamento.
Reservas	Beds24 (fonte de verdade); data/reservations/ guarda o estado de cada pagamento.	Consultar no Beds24 e no Stripe.
Configuração e contas	api/config.php e admin/users.json, no servidor (nunca no git).	Guardar uma cópia fora do servidor, em local seguro.
Cópias do alojamento	Painel do alojamento (p. ex. cPanel → Backups).	Recomendado: cópias diárias de todo o site.
Arquivo local	_arquivo/, só no computador de desenvolvimento: ficheiros sem uso no site.	Ver _arquivo/LEIA-ME.md.

O mesmo procedimento e a mesma tabela estão em `DEPLOY.md`, no repositório.

Depois de publicar

- **Monitorização** de disponibilidade (uptime) e alertas de erros PHP.
- **Emails:** SPF, DKIM e DMARC no domínio para os emails de reserva não irem para spam.
- Rever periodicamente o refresh token do Beds24 (expira após 30 dias sem uso).

Pré-visualização local: enquanto não há alojamento, o site pode ser visto com `php -S 127.0.0.1:8080 -t .` na pasta do projeto.

6. SEO técnico (antes dos anúncios)

Os anúncios levam tráfego para páginas que o Google também avalia (qualidade da página de destino). Estes pontos não existem ainda e convém fazê-los antes de lançar campanhas:

- **title** e **meta description** próprios em cada página (hoje não há meta description).
- Etiquetas **Open Graph** (imagem e texto ao partilhar em redes sociais / WhatsApp).
- **favicon** e ícone para telemóvel (não existem).
- **robots.txt** (bloquear /admin/, /api/, /data/) e **sitemap.xml** com as páginas e os 60 apartamentos (`property.html?p=<slug>`).
- **Google Search Console**: verificar o domínio e submeter o sitemap.
- **Google Business Profile** da empresa (morada, telefone, fotos, avaliações).
- **Limitação a conhecer:** a página de cada apartamento é um modelo único preenchido por JavaScript, e a mudança de língua também é feita por JavaScript no mesmo URL. O Google consegue ler JavaScript, mas EN e FR não têm URLs próprios. Para SEO internacional forte, considerar páginas por língua ou geração no servidor.
- Imagens: 17 fotos de 3 apartamentos vêm diretamente do Airbnb (a0.muscache.com) — descarregar para img/.

7. Google Ads e medição

O que já está preparado

- Aviso de cookies com aceitar/rejeitar em todas as páginas, que guarda a escolha em `localStorage["tmh_cookie_consent"]` e expõe `window.TMHConsent` (`isAccepted()`, `onChange(fn)`, `open()`) e o evento `tmh-cookie-consent`.
- Página de confirmação `reserva.html`, que sabe o estado, a referência e o valor da reserva — o ponto natural para registar a conversão.

Contas a criar / ligar

- Google Ads e Google Analytics 4 (ligados entre si); opcionalmente Google Tag Manager.
- Google Search Console e Google Business Profile (ver secção 6).

Consentimento (Consent Mode v2)

No Espaço Económico Europeu, o Google exige o Consent Mode v2 para usar dados de anúncios. A etiqueta do Google deve arrancar com tudo recusado e ser atualizada pelo aviso de cookies do site. Exemplo, a pôr no <head> de todas as páginas, antes do script do gtag:

```
<script>
  window.dataLayer = window.dataLayer || [];
  function gtag(){ dataLayer.push(arguments); }
  gtag('consent', 'default', { ad_storage: 'denied', ad_user_data: 'denied',
       ad_personalization: 'denied', analytics_storage: 'denied' });
</script>
<script async src="https://www.googletagmanager.com/gtag/js?id=AW-XXXXXXXXX"></script>
<script>
  gtag('js', new Date()); gtag('config', 'AW-XXXXXXXXX'); gtag('config', 'G-XXXXXXXXXX');
</script>
```

E, depois de js/cookie-consent.js, ligar a escolha do visitante:

```
function tmhApplyConsent(accepted) {
  var v = accepted ? 'granted' : 'denied';
  gtag('consent', 'update', { ad_storage: v, ad_user_data: v,
       ad_personalization: v, analytics_storage: v });
```

```
}
tmhApplyConsent(TMHConsent.isAccepted());
TMHConsent.onChange(function (s) { tmhApplyConsent(s === 'accepted'); });
```

> O texto do aviso de cookies fala apenas em "medir a audiência". Com anúncios, tem de mencionar também cookies de publicidade, e a política de cookies tem de listar os do Google.

Conversões

Em `js/booking-result.js`, quando a reserva fica confirmed, enviar a conversão uma única vez (guardar a referência em sessionStorage para não duplicar ao recarregar):

```
gtag('event', 'conversion', {
  send_to: 'AW-XXXXXXXXX/YYYYYYYY',   // ação de conversão "Reserva confirmada"
  value: r.total, currency: 'EUR', transaction_id: r.id
});
```

- Conversões secundárias úteis: pedido de reserva sem pagamento (status request), cliques no telefone e no email, envio do formulário de proprietários.
- Validar tudo com o Tag Assistant do Google antes de ativar campanhas.

Campanhas (sugestão de arranque)

- **Pesquisa — marca** ("TheMajord'Home") e **genérica** ("alojamento local Porto Baixa", "apartamento Ribeira"…), com as páginas dos apartamentos como destino.
- **Proprietários:** campanha separada para "gestão de alojamento local Porto" → páginas de serviços.
- Verificar se a conta Beds24 permite ligação ao Google (Vacation Rentals / Hotel Center) para aparecer nos resultados de alojamento do Google.
- Só ativar campanhas de reservas depois de o pagamento estar ligado e testado.

8. Pendentes e pontos de atenção

Os pontos legais devem ser confirmados com um jurista ou contabilista; a lista indica o que verificar, não substitui esse aconselhamento.

Legal (crítico antes de aceitar pagamentos e anúncios)

Ponto	Situação
Aviso legal	legal-notice.html é um modelo francês (leis francesas, CNIL) e menciona outro site (collenia.com). Refazer para Portugal, em PT/EN/FR.
Política de privacidade	Não existe. Necessária (RGPD): reservas recolhem nome, email, telefone; dados passam pelo Beds24, Stripe e Google.
Política de cookies	Não existe. Necessária, sobretudo com Google Analytics/Ads.
Termos e condições de reserva	Não existem: política de cancelamento e reembolso, sinal, check-in/out, regras da casa, responsabilidade.
Registo de alojamento local	Verificar a obrigação de mostrar o nº de registo (RNAL) de cada apartamento no anúncio.
Livro de Reclamações	Verificar a obrigação de ligação ao Livro de Reclamações Eletrónico e a informação sobre resolução alternativa de litígios.
Identificação da empresa	Nome legal, NIF e morada no site.
Subcontratantes (RGPD)	Acordos de tratamento de dados com Beds24, Stripe, Google e o alojamento.
Direitos de imagem	Fotos enviadas para os serviços eram de outras empresas/bancos de imagens; confirmar direitos. Fotos e avaliações vindas do Airbnb: confirmar que podem ser usadas.

Conteúdo

- As 4 páginas de serviços estão só em português (faltam EN/FR).
- As galerias dos serviços usam fotos de apartamentos como provisórias (exceto concierge-2.jpg).
- Nenhum apartamento tem preço "a partir de": aparece "Preço sob consulta" em todo o lado até o Beds24 estar ligado.
- Os 60 apartamentos estão todos marcados como destaque (pedido da cliente) — a página inicial mostra os 60.
- Não existe nenhum apartamento chamado "Pretty Porto" (havia um pedido para o apagar).

Técnico

- `js/reviews-data.js` começa como lista vazia mas o código trata-o como objeto (o painel corrige ao aprovar a primeira avaliação).
- Os apartamentos têm um campo "price" sem uso (pode ser retirado dos dados).
- Analytics e testes automáticos não existem.

9. Checklist de lançamento

Antes de publicar

- ☐ Textos legais refeitos e publicados (aviso legal, privacidade, cookies, termos de reserva).
- ☐ Alojamento com PHP 8.1+ e HTTPS; domínio a apontar.
- ☐ Repositório publicado sem .claude/, _arquivo/ e ficheiros soltos.
- ☐ api/config.php criado; conta do painel criada e password trocada.
- ☐ Permissões de escrita do PHP confirmadas; display_errors desligado.
- ☐ Meta descriptions, favicon, robots.txt, sitemap.xml.

Reservas

- ☐ Refresh token do Beds24 configurado; 60 roomId preenchidos.
- ☐ Pontos VERIFICAR do api/beds24.php confirmados no Swagger.
- ☐ Stripe em modo teste: reserva completa com o cartão 4242…, webhook recebido, reserva confirmada no Beds24.
- ☐ Expiração testada (reserva cancelada e datas libertadas).
- ☐ Emails a sair por SMTP; SPF/DKIM/DMARC configurados.
- ☐ Chaves live; interruptores em 'live'; texto property.finePrint atualizado.

Google

- ☐ Search Console verificado e sitemap submetido; Google Business Profile.
- ☐ GA4 + Google Ads com Consent Mode v2 ligado ao aviso de cookies.
- ☐ Conversão "Reserva confirmada" validada com o Tag Assistant.
- ☐ Texto do aviso de cookies e política de cookies a mencionar publicidade.

Depois

- ☐ Cópias diárias do alojamento ativas; api/config.php e admin/users.json guardados fora do servidor.
- ☐ Monitorização de disponibilidade e erros.
- ☐ Rever as primeiras reservas reais no Beds24 e no Stripe.